



Universidad Europea

UNIVERSIDAD EUROPEA DE MADRID

ESCUELA DE ARQUITECTURA, INGENIERÍA Y DISEÑO

GRADO EN INGENIERÍA INFORMÁTICA

Propuesta de proyecto

Zona Residencial Simulada

AureaTech

Daniel De Abajo Nacarino

Paula Gómez Lucas

Franco Nahuel Zimmermann

Pablo Serrano Tirado

CURSO 2025/26

Septiembre, 2025

Tabla de Contenidos

1. Introducción.....	3
1.1. Estado del Arte.....	3
1.2. Justificación.....	5
1.3. Objetivos del proyecto.....	5
2. Planificación del Proyecto.....	8
2.1. Descripción del Proyecto.....	8
2.2. Historias de Usuario.....	9
2.3. Requisitos.....	10
2.4. Desglose de tareas del proyecto.....	12
2.5. Diagrama de Gantt.....	28
2.6. Presupuesto.....	29
3. Diseño del proyecto.....	32
4. Resultados esperados.....	33
5. Referencias.....	34



1. Introducción

1.1. Estado del Arte

1.1.1. Fotorresistor (LDR)

Funcionamiento

Un fotorresistor o LDR es un componente electrónico cuya resistencia disminuye al aumentar la intensidad de luz incidente. Su principio se basa en el efecto fotoconductor de materiales como el sulfuro de cadmio (CdS): cuando la luz incide, se liberan electrones, lo que permite que la corriente fluya con mayor facilidad.

Características principales

- En oscuridad → resistencia muy alta ($\approx 1\text{ M}\Omega$).
- Con luz intensa → resistencia muy baja ($\approx 100\ \Omega$).
- Dispositivo fotoeléctrico sensible a la luz.
- Tiempo de respuesta: décimas de segundo (no apto para cambios muy rápidos).
- Fácil de usar (dos terminales, se conecta como un resistor).
- Bajo costo y gran disponibilidad.
- Superficie sensible relativamente amplia.

1.1.2. Sensor de distancia (HC-SR04 Ultrasonidos)

Funcionamiento

El HC-SR04 es un sensor de distancias por ultrasonidos capaz de detectar objetos y calcular la distancia a la que se encuentra en un rango de 2 a 450 cm. El sensor funciona por ultrasonidos y contiene toda la electrónica encargada de hacer la medición.



Características principales

- Frecuencia de trabajo: 40 KHz
- Rango máximo de detección: 4.5 m
- Rango mínimo de detección: 1.7 cm
- Dispositivo ultrasónico que mide distancia mediante emisión y recepción de ondas sonoras.
- Fácil de usar (cuatro terminales: Vcc, GND, Trigger y Echo, con comunicación sencilla por pulsos).
- Bajo costo y gran disponibilidad.
- Ángulo de detección estrecho (aproximadamente 15 grados), con precisión en distancias de 2 cm a 400 cm.

1.1.3. DHT22. Sensores de temperatura y humedad

Funcionamiento

DHT22 son sensores digitales que entregan medidas de temperatura y humedad ambientales simultáneamente. Además, incluyen un procesador interno que convierte las señales analógicas de los sensores en señales digitales listas para su uso.

Características principales

- Tiempo de respuesta: alrededor de 2 segundos (no apto para cambios muy rápidos).
- Rango de humedad: 0% a 100% RH
- Precisión humedad: $\pm 2\%$ RH
- Rango de temperatura: $-40\text{ }^{\circ}\text{C}$ a $+80\text{ }^{\circ}\text{C}$
- Precisión temperatura: $\pm 0.5\text{ }^{\circ}\text{C}$
- Salida de datos: Señal digital de un solo hilo
- Frecuencia de muestreo: 0.5 Hz (una lectura cada 2 segundos)



1.2. Justificación

Las demandas de las comunidades residenciales evolucionan hacia entornos más seguros, cómodos y sostenibles, este proyecto representa una solución moderna, eficiente:

- **Mejora de la seguridad comunitaria:** A través de sensores de proximidad, cámaras y sistemas de alarma, se incrementa la capacidad de respuesta ante situaciones potencialmente peligrosas (presencias no autorizadas, incendios, etc.), brindando tranquilidad a los residentes.
- **Optimización del consumo energético:** El sistema de iluminación inteligente, basado en sensores de movimiento y condiciones lumínicas, permite un uso eficiente de la energía, encendiendo únicamente las farolas necesarias en momentos clave. Esta medida no solo reduce el gasto energético, sino que contribuye a una gestión sostenible del entorno.
- **Reducción de gastos económicos:** Gracias a los sensores de humo se podrá intervenir antes en casos de incendios reduciendo los daños económicos.
- **Control centralizado y accesible:** La aplicación móvil y web desarrollada proporciona acceso a la información de los sensores correspondiente a los usuarios, además de un administrador y Técnico de mantenimiento para gestionar la aplicación.

1.3. Objetivos del proyecto

Los objetivos generales del proyecto consisten en

- Crear un sistema capaz de monitorear y controlar una zona residencial mediante sensores y actuadores que sirva para mejorar la comodidad, seguridad y sostenibilidad de la comunidad.

Estos objetivos generales llevan a la necesidad de definir los siguientes objetivos específicos:

- Hacer que las farolas se enciendan de noche cuando pasa algo o alguien próximo a ellas mediante el sensor de distancia.



- Diseñar una aplicación móvil y web mediante "Flet" vinculada con el sistema de sensores y actuadores.
- Hacer un login para los usuarios donde se almacenarán en una base de datos.
- Crear un backend en Python y usar archivos json para almacenar la información necesaria para su correcto funcionamiento.
- Crear un panel en el que el administrador sea capaz de gestionar los usuarios.
- Crear un panel en el que el administrador pueda controlar de forma remota los actuadores.
- Crear un panel general en el que se visualice la temperatura y humedad a que pueda acceder el usuario vecino medidos mediante el sensor DHT11/DHT22.
- Crear un panel en el que el usuario administrador y el usuario técnico puedan ver el historial de acciones del sistema.

Opcionales:

- Implementar un anemómetro digital y que el usuario pueda ver las mediciones desde la app.
- Crear un sistema de alarma de incendios, mediante implementación de sensores de humo y sistema de aviso..
- Diseñar un sistema de puerta automática de garaje con apertura mediante lector de matrículas (además de manual como sistema de seguridad) mediante el uso del sensor OV2640.



2. Planificación del Proyecto

2.1. Descripción del Proyecto

El proyecto Zona Residencial Simulada, desarrollado por el equipo Aurea Tech, tiene como propósito diseñar e implementar un sistema domótico inteligente orientado a la monitorización y control de una zona residencial simulada.

El sistema integra sensores ambientales y de proximidad —como el DHT22 para temperatura y humedad, el LDR para detección lumínica y el HC-SR04 para medición de distancia— junto con actuadores controlados a través de una aplicación web y móvil desarrollada en Flet (Python). La arquitectura del software sigue el patrón Modelo–Vista–Controlador (MVC), garantizando modularidad, mantenibilidad y separación lógica de componentes.

El objetivo general del sistema es optimizar la eficiencia energética, la seguridad y el confort de los residentes mediante un control automatizado o remoto de los dispositivos. Los datos capturados por los sensores se almacenan en ficheros JSON, permitiendo su trazabilidad, portabilidad y uso en entornos educativos.

El sistema contempla tres roles principales de usuario:

- Administrador: gestiona usuarios, sensores y actuadores.
- Técnico de mantenimiento: supervisa lecturas e historial del sistema.
- Vecino o usuario general: accede a las lecturas básicas (temperatura, humedad y alertas).

Cada rol dispone de interfaces diferenciadas dentro de una aplicación unificada, adaptada a distintas resoluciones de pantalla y coherente en su diseño visual.

Como ampliaciones opcionales, se plantea la incorporación de módulos adicionales, tales como un sistema de alarma de incendios, apertura automática de garaje mediante cámara OV2640, y anemómetro



digital para registro de viento, fortaleciendo la escalabilidad del sistema y su integración dentro de un entorno IoT educativo y sostenible.

En conjunto, este proyecto constituye una implementación práctica de ingeniería del software y sistemas embebidos, enfocada en la automatización residencial mediante la integración de hardware, software y diseño de interfaces bajo los principios de eficiencia, sostenibilidad y control inteligente.

2.2. Historias de Usuario

Como usuario general debo poder/tener:

- Debo poder iniciar sesión.

Como administrador debo poder/tener:

- Visualizar la medición de los sensores y el historial.
- Controlar los actuadores en remoto.
- Ver mi identificador y rol de usuario en todo momento.
- Gestionar usuarios.
- Debo tener acceso a la base de datos.

Como mantenimiento debo poder/tener:

- Ver la medición de sensores y su historial.
- Ver mi identificador y rol de usuario en todo momento.
- Controlar los actuadores en remoto.

Como usuario común/vecino debo poder/tener:

- Ver los datos básicos de la app (temp-humedad- ...).
- Visualizar un cuadro de alertas.



2.3. Requisitos

2.3.1. Requisitos obligatorios

Implementación

- R1. Se usa patrón Modelo-Vista-Controlador
- R2. Se usa FLET (para Python 3)
- R3. Debe funcionar y compilar en el IDE PyCharm.
- R4. El proyecto debe incluir las librerías externas que necesite, y las rutas deben ser relativas al proyecto para que este sea portable.

Visualización

- R5. Cada vista debe mostrar de alguna manera el nombre y rol del usuario.
- R6. La aplicación debe verse correctamente en diferentes resoluciones de pantalla

Login

- R7. Debe desarrollarse una única aplicación para todos los roles de usuario.
- R8. Debe tener posibilidad de registro y no solo de acceso.
- R9. Cada rol diferente debe llevar a una vista diferente

Usuarios, roles y sensores

- R10. El sistema debe involucrar al menos tres tipos de usuarios diferentes, con distintos roles, pero que se relacionen entre sí. Por ejemplo, en una plataforma deportiva habrá jugadores, médicos y entrenadores.
- R11. Uno de los roles de usuario tendrá asociados al menos 3 tipos de sensores diferentes. Por ejemplo, a un deportista se le monitoriza localización GPS, frecuencia cardíaca y posición.



Almacenamiento de datos

- R12. Toda la aplicación debe funcionar sobre ficheros de texto en formato de JSON
- R13. Cada grupo debe crear sus propios ficheros, deben cumplir que:
 - Debe haber al menos 9 usuarios principales (ej: deportistas, pacientes)
 - Al menos uno de los usuarios principales debe tener más de un supervisor (ej: entrenador, familiar, agente)
 - Debe haber al menos 4 supervisores
 - Al menos uno de ellos debe tener más de un usuario principal
 - Debe haber al menos 2 usuarios clínicos (los que analizarán los datos de los sensores, ej: médico de cabecera, médico del club, fisioterapeuta, analista)
 - Debe haber al menos 1 sensor por usuario principal
 - Debe haber al menos información de 3 tipos de sensores diferentes
 - Cada sensor debe mostrar al menos medidas en 3 días diferentes
 - Los datos deben ser realistas: el DNI debe estar formado por números y una letra, el número de teléfono debe tener 9 números, los nombres y las direcciones deben tener visos de veracidad, etc.

2.3.2. Requisitos opcionales

Visualización

- RO1. El equipo debe tener un nombre y un logo, que aparecerá en la aplicación.
- RO2. Las distintas ventanas deben tener una coherencia estética.



- RO3. Se debe visualizar de manera intuitiva los datos de los usuarios y de los sensores, incluyendo información histórica de los sensores.

2.4. Desglose de tareas del proyecto

En la siguiente tabla se recoge la planificación completa del desarrollo del sistema, estructurada en los cuatro Sprints definidos según la metodología SCRUM.

Cada sprint tiene una duración efectiva de 12 días (días 0 a 11), seguida de un intervalo de dos días de buffer destinado a revisión, integración y preparación de la siguiente iteración.

La planificación se ha elaborado siguiendo un modelo de concurrencia equilibrada, que permite la ejecución simultánea de tareas independientes siempre que no exista dependencia directa entre ellas y que la carga de trabajo de cada integrante del equipo se mantenga dentro de los márgenes razonables.

Descripción de las columnas

- **Código:** Identificador único asignado a cada tarea. Está compuesto por un prefijo que indica la naturaleza de la tarea y un número que refleja el sprint y el orden en el que aparece dentro de dicho sprint. Por ejemplo, el código RF203 corresponde a una tarea de tipo Requisito Funcional (RF), perteneciente al Sprint 2 y registrada como la tercera tarea de dicho bloque. Los prefijos utilizados se definen a continuación:

Prefijo	Tipo de tarea	Descripción
INF	Infraestructura	Configuración inicial del entorno, repositorios y componentes base necesarios para el desarrollo.
RNF	Requisito No Funcional	Implementaciones relacionadas con estructura, rendimiento, compatibilidad y buenas prácticas del sistema.



UI	Interfaz de Usuario	Diseño y desarrollo de la interfaz visual mediante la librería Flet y los mockups definidos.
RF	Requisito Funcional	Desarrollo de funcionalidades principales, controladores, vistas y gestión de roles y sensores.
BD	Base de Datos	Definición, lectura y escritura en ficheros JSON y operaciones de persistencia de datos.
TES	Testing	Validación, pruebas funcionales y verificación de los componentes del sistema.
DOC	Documentación	Elaboración de actas, informes, documentación técnica y entrega de resultados.

- **Detalles:** Describe brevemente la acción concreta que se debe realizar, de acuerdo con lo definido en el Product Backlog.
- **Persona asignada:** Indica el miembro del equipo responsable de la ejecución de la tarea. La distribución busca equilibrar la carga de trabajo y mantener la continuidad de las responsabilidades a lo largo de los sprints.
- **Prioridad:** Determina el nivel de criticidad o relevancia de cada tarea en función de los objetivos del sprint. Se emplean principalmente las categorías Imprescindible e Importante.
- **Inicio (día):** Señala el día, dentro del intervalo de duración del sprint (0-11), en que se inicia la tarea. El día 0 corresponde al kickoff del sprint, y los valores posteriores indican la progresión temporal dentro de la iteración. Las tareas pueden desarrollarse de forma simultánea si no existen dependencias que lo impidan.
- **Duración (días):** Representa la duración total estimada de la tarea, expresada en días de trabajo activo. Esta estimación se ha calculado



a partir del coste temporal previsto en horas, redondeado a jornadas completas.

- **Dependencias:** Especifica las tareas cuya finalización resulta necesaria para el inicio de la tarea en cuestión. Por ejemplo, las tareas de tipo TES (Testing) dependen de la finalización de las RF y BD, mientras que las tareas DOC suelen depender de la culminación de las pruebas y validaciones del sprint.

Cód.	Detalles	Persona Asignada	Prioridad	Inicio (día)	Duración (días)	Dependencias
RNF101	Revisar carpetas y rutas para garantizar modularidad.	Paula	Imprescindible	0	1	—
UII02	Implementar formulario de registro en Flet con validación de datos.	Daniel	Imprescindible	0	1	—
UII03	Diseñar alertas en la interfaz de login.	Franco	Imprescindible	0	1	—
RF104	Implementar clase Usuario con verificación de credenciales.	Pablo	Imprescindible	0	1	—
RF105	Añadir campo rol al esquema JSON y clase Usuario.	Paula	Imprescindible	1	1	—
RF106	Implementar lógica de login con verificación y redirección.	Daniel	Imprescindible	1	1	—
RNF107	Implementar botón “Cerrar sesión” y limpieza del estado.	Franco	Importante	1	1	—
RF108	Desarrollar sistema de enrutamiento según tipo de usuario.	Pablo	Importante	1	1	—
RF109	Crear y escribir usuarios en usuarios.json.	Paula	Importante	2	1	—



Cód.	Detalles	Persona Asignada	Prioridad	Inicio (día)	Duración (días)	Dependencias
RNF110	Comprobar rutas relativas y compatibilidad en distintos SO.	Franco	Interesante	2	1	—
RNF111	Implementar recuperación simulada de contraseña.	Daniel	Imprescindible	2	1	—
RF112	Añadir excepciones controladas en el backend.	Paula	Importante	3	1	—
RF113	Evaluar flujo de login con casos positivos/negativos.	Franco	Importante	3	1	—
RF114	Diseñar layout minimalista de login y registro.	Pablo	Imprescindible	2	1	—
BD115	Guardar fecha/hora de inicio de sesión en logs.json.	Paula	Importante	4	1	—
RNF116	Crear estructura de funciones y comentarios.	Daniel	Interesante	3	1	—
UI117	Mostrar usuario actual en interfaz.	Franco	Imprescindible	4	1	—
BD118	Crear función “Actualizar perfil” en JSON.	Pablo	Importante	3	1	—
BD119	Escribir test básico de lectura y escritura de JSON.	Paula	Importante	5	1	—
RNF120	Revisar dependencias y librerías.	Daniel	Interesante	4	1	—
RNF121	Implementar pantalla de confirmación de alta.	Paula	Imprescindible	6	1	—



Cód.	Detalles	Persona Asignada	Prioridad	Inicio (día)	Duración (días)	Dependencias
RNF122	Simular intentos fallidos y bloqueo temporal.	Franco	Imprescindible	5	1	—
RNF123	Implementar config.py con rutas relativas.	Daniel	Importante	5	1	—
UI124	Añadir mensajes de éxito/error en la interfaz.	Pablo	Importante	4	1	—
TES125	Añadir botón de "Limpiar logs" para testing.	Paula	Interesante	7	1	BD115, BD118, BD119, BD126, BD130, RF104, RF105, RF106, RF108, RF109, RF112, RF113, RF114, RF127, RF129, RF134, RF136, RF137, RF138
BD126	Añadir try/except a las operaciones JSON.	Daniel	Imprescindible	6	1	—
RF127	Implementar autoguardado tras registro.	Franco	Importante	6	1	—
RNF128	Validar visibilidad y accesibilidad de los campos.	Pablo	Importante	5	1	—
RF129	Verificar paleta de colores y tipografía del login.	Paula	Importante	8	1	—
BD130	Guardar fallos en errores.json.	Daniel	Interesante	7	1	—
RNF131	Contar e informar tras tres fallos consecutivos.	Franco	Interesante	7	1	—
RNF132	Validar campos antes de enviar formulario.	Pablo	Importante	6	1	—



Cód.	Detalles	Persona Asignada	Prioridad	Inicio (día)	Duración (días)	Dependencias
RNF133	Implementar guardado inmediato tras modificación.	Paula	Imprescindible	9	1	—
RF134	Test funcional de login completo.	Daniel	Importante	8	1	—
RNF135	Comprobar ejecución en distintos equipos.	Franco	Importante	8	1	—
RF136	Mostrar mensaje según rol tras iniciar sesión.	Pablo	Interesante	7	1	—
RF137	Crear script de test automático de login.	Paula	Interesante	10	1	—
RF138	Generar JSON con 3 usuarios y distintos roles.	Daniel	Interesante	9	1	—
RNF139	Validar flujo completo antes del Review.	Franco	Importante	9	1	—
DOC140	Documentar tareas completadas en el acta.	Pablo	Importante	8	1	TES125
INF201	Documentar y configurar los tipos de sensores (temp, humedad, luz, distancia).	Paula	Imprescindible	0	1	—
RNF202	Implementar función de simulación periódica de datos.	Daniel	Imprescindible	0	1	—
UI203	Diseñar el panel de datos en Flet con refresco en vivo.	Franco	Imprescindible	0	1	—
RF204	Crear botones de control ON/OFF para actuadores simulados.	Pablo	Imprescindible	0	1	—



Cód.	Detalles	Persona Asignada	Prioridad	Inicio (día)	Duración (días)	Dependencias
BD205	Añadir escritura de logs de cambios de estado en historial.json.	Paula	Imprescindible	1	1	—
RNF206	Definir campos coherentes (sensor, valor, fecha, hora).	Daniel	Importante	1	1	—
RNF207	Programar avisos visuales para valores fuera de rango.	Franco	Importante	1	1	—
RNF208	Mantener formato y paleta según guía AureaTech.	Pablo	Importante	1	1	—
RNF209	Permitir habilitar/deshabilitar sensores desde el panel.	Paula	Interesante	2	1	—
RNF210	Implementar slider o input para frecuencia de refresco.	Daniel	Importante	2	1	—
RF211	Medir latencia de refresco en UI y registro JSON.	Franco	Importante	2	1	—
RNF212	Ajustar rangos de simulación para lecturas realistas.	Pablo	Interesante	2	1	—
RNF213	Comprobar que los valores cambian en cada actualización.	Paula	Imprescindible	3	1	—
RNF214	Usar async o time para intervalos no bloqueantes.	Daniel	Importante	3	1	—
RNF215	Implementar log rotativo (solo 50 últimas lecturas activas).	Franco	Importante	3	1	—
RNF216	Añadir etiquetas de °C, %, cm, Lux a cada sensor.	Pablo	Interesante	3	1	—



Cód.	Detalles	Persona Asignada	Prioridad	Inicio (día)	Duración (días)	Dependencias
BD217	Añadir botón “Exportar a JSON” desde el dashboard.	Paula	Imprescindible	4	1	—
RF218	Formatear registros con timestamp y rol de usuario.	Daniel	Importante	4	1	—
RNF219	Añadir modo “resumen” del dashboard con datos promedio.	Franco	Interesante	4	1	—
RNF220	Simular fallos aleatorios con aviso visual.	Pablo	Importante	4	1	—
RNF221	Simular 1h de funcionamiento continuo y comprobar logs.	Paula	Imprescindible	5	1	—
RF222	Permitir reordenar columnas en la vista.	Daniel	Importante	5	1	—
RNF223	Separar funciones comunes de lectura/escritura.	Franco	Interesante	5	1	—
RNF224	Implementar botón “Activar todos” y “Desactivar todos”.	Pablo	Importante	5	1	—
RNF225	Mejorar la disposición de tarjetas con colores adaptativos.	Paula	Interesante	6	1	—
BD226	Añadir validaciones antes de sobrescribir JSON.	Daniel	Importante	6	1	—
RNF227	Colorear tarjeta cuando el valor cambie.	Franco	Imprescindible	6	1	—
RNF228	Separar dashboard en componentes reutilizables.	Pablo	Importante	6	1	—



Cód.	Detalles	Persona Asignada	Prioridad	Inicio (día)	Duración (días)	Dependencias
RNF229	Verificar funcionalidad integral de sensores y actuadores.	Paula	Imprescindible	7	1	—
RNF230	Añadir botón de pausa/reanudación de lecturas.	Daniel	Importante	7	1	—
RNF231	Implementar modo oscuro opcional.	Franco	Interesante	7	1	—
UI232	Permitir editar configuración de sensor desde interfaz.	Pablo	Importante	7	1	—
RNF233	Registrar errores, pausas y cambios de frecuencia.	Paula	Imprescindible	8	1	—
RNF234	Organizar módulos en subcarpetas por tipo de sensor.	Daniel	Importante	8	1	—
RNF235	Añadir colapsado de secciones del dashboard.	Franco	Interesante	8	1	—
RF236	Validar visualización, control y logs simultáneamente.	Pablo	Importante	8	1	—
RNF237	Simular hilos o tareas asíncronas para rendimiento.	Paula	Interesante	9	1	—
INF238	Ejecutar pruebas en distintos entornos locales.	Daniel	Importante	9	1	—
DOC239	Registrar avances y bloqueos en acta diaria.	Franco	Imprescindible	9	1	—
RNF240	Verificar cumplimiento del objetivo del sprint.	Pablo	Importante	9	1	—



Cód.	Detalles	Persona Asignada	Prioridad	Inicio (día)	Duración (días)	Dependencias
RNF301	Establecer el esquema de paneles (Admin, Técnico, Vecino).	Franco	Imprescindible	0	1	—
RF302	Desarrollar CRUD (añadir, editar, eliminar) en JSON.	Paula	Imprescindible	0	2	—
RNF303	Crear panel de Técnico con visualización de datos y logs.	Daniel	Imprescindible	0	1	—
RF304	Desarrollar panel de Vecino con lecturas simplificadas.	Pablo	Imprescindible	0	1	—
RF305	Implementar tabla dinámica de historial con scroll y paginación.	Paula	Imprescindible	2	1	—
RNF306	Añadir filtros por tipo de sensor y fecha.	Daniel	Importante	1	1	—
RNF307	Implementar buscador textual en historial.	Franco	Importante	1	1	—
RF308	Crear middleware que controle acceso según rol.	Pablo	Importante	1	1	—
RNF309	Añadir formulario editable dentro del panel de Admin.	Paula	Importante	3	1	—
RF310	Mostrar el rol en la esquina superior del panel.	Daniel	Interesante	2	1	—
RF311	Añadir botón “Exportar JSON” en su vista.	Franco	Importante	2	1	—
RNF312	Guardar en logs cada operación CRUD.	Pablo	Imprescindible	2	1	—



Cód.	Detalles	Persona Asignada	Prioridad	Inicio (día)	Duración (días)	Dependencias
RNF313	Mantener coherencia visual entre los tres paneles.	Paula	Importante	4	1	—
RNF314	Añadir botón “Actualizar ahora” en su panel.	Daniel	Importante	3	1	—
RNF315	Incluir mini gráficas para evolución de temperatura/humedad.	Franco	Interesante	3	1	—
RNF316	Añadir botón de eliminación con confirmación.	Pablo	Imprescindible	3	1	—
RNF317	Mostrar media, min y max por sensor.	Paula	Importante	5	1	—
RNF318	Implementar navegación por menú lateral.	Daniel	Importante	4	1	—
RNF319	Verificar rutas de navegación entre paneles.	Franco	Imprescindible	4	1	—
RNF320	Mostrar columna “Último cambio realizado por” en el historial.	Pablo	Interesante	4	1	—
RF321	Separar vistas de paneles en componentes reutilizables.	Paula	Importante	6	1	—
BD322	Crear logs_admin.json con acciones relevantes.	Daniel	Imprescindible	5	1	—
RNF323	Mostrar estado ON/OFF en el panel técnico.	Franco	Importante	5	1	—
RNF324	Resaltar datos en rojo si superan límites.	Pablo	Importante	5	1	—



Cód.	Detalles	Persona Asignada	Prioridad	Inicio (día)	Duración (días)	Dependencias
RNF325	Revisar contraste de color y tamaño de texto.	Paula	Interesante	7	1	—
RF326	Simular acceso de los tres tipos de usuarios.	Daniel	Importante	6	1	—
RNF327	Añadir ordenación por fecha o tipo de evento.	Franco	Interesante	6	1	—
TES328	Crear tests automáticos de cada panel.	Pablo	Importante	6	1	BD322, RF302, RF304, RF305, RF308, RF310, RF311, RF321, RF326, RF333, RF335, RF338
RNF329	Implementar visión global del sistema.	Paula	Importante	8	1	—
RNF330	Mostrar hora y fecha de última lectura.	Daniel	Imprescindible	7	1	—
RNF331	Permitir orden ascendente/descendente en tabla.	Franco	Importante	7	1	—
RNF332	Opción de descarga de lecturas personales.	Pablo	Interesante	7	1	—
RF333	Añadir gestión de errores por rol (permisos, datos).	Paula	Importante	9	1	—
RNF334	Probar resistencia a interrupciones simuladas.	Daniel	Imprescindible	8	1	—
RF335	Mostrar totales (usuarios, sensores, eventos).	Franco	Interesante	8	1	—



Cód.	Detalles	Persona Asignada	Prioridad	Inicio (día)	Duración (días)	Dependencias
DOC336	Documentar estado diario de los paneles.	Pablo	Importante	8	1	TES328
RNF337	Validar estructura de todos los ficheros de datos.	Paula	Importante	10	1	—
RF338	Implementar script de test sobre roles y accesos.	Daniel	Interesante	9	1	—
RNF339	Validar paneles, historial y permisos antes de entrega.	Franco	Importante	9	1	—
DOC340	Redactar acta final y registrar avances completados.	Pablo	Importante	9	1	TES328
RNF401	Revisar compatibilidad entre los paneles, dashboard y backend.	Daniel	Imprescindible	0	1	—
RNF402	Planificar la secuencia de integración y resolver conflictos.	Paula	Imprescindible	0	1	—
UI403	Conectar las vistas de Flet con la lógica JSON persistente.	Paula	Imprescindible	1	1	—
RNF404	Optimizar tiempos de lectura/escritura y refresco de UI.	Franco	Importante	0	1	—
RNF405	Unificar estilos, tipografías y espaciados en todos los paneles.	Pablo	Importante	0	1	—
RF406	Verificar autenticación, roles, simulación y control de actuadores.	Daniel	Imprescindible	1	1	—



Cód.	Detalles	Persona Asignada	Prioridad	Inicio (día)	Duración (días)	Dependencias
RNF407	Depurar excepciones y logs de errores durante la integración.	Paula	Importante	2	1	—
RNF408	Ejecutar simulación de sensores durante varias horas.	Franco	Importante	1	1	—
RNF409	Validar el middleware de acceso en todos los paneles.	Pablo	Importante	1	1	—
RNF410	Ajustar colores y tamaños de botones, márgenes y tipografía.	Paula	Interesante	3	1	—
RNF411	Comprobar que las funciones previas siguen operativas.	Daniel	Imprescindible	2	1	—
TES412	Testear consistencia del historial en múltiples operaciones.	Franco	Importante	2	1	BD424, BD428, RF406, RF429, RF436
TES413	Ejecutar script de validación de flujos principales.	Pablo	Importante	2	1	BD424, BD428, RF406, RF429, RF436
RNF414	Añadir resumen gráfico de lecturas y acciones (gráficas simples).	Paula	Interesante	4	1	—
DOC415	Redactar memoria técnica con arquitectura y decisiones.	Daniel	Imprescindible	3	1	TES412, TES413, TES416, TES433, TES440
TES416	Realizar sesión de prueba final antes de la entrega.	Franco	Importante	3	1	BD424, BD428, RF406, RF429, RF436



Cód.	Detalles	Persona Asignada	Prioridad	Inicio (día)	Duración (días)	Dependencias
RNF417	Comprobar que los menús y rutas funcionan sin errores.	Pablo	Importante	3	1	—
RNF418	Eliminar duplicaciones entre módulos de paneles.	Paula	Interesante	5	1	—
RNF419	Revisar checklist completo de requisitos cumplidos.	Daniel	Imprescindible	4	1	—
RNF420	Añadir notificaciones cuando se produzca un fallo crítico.	Franco	Importante	4	1	—
RNF421	Añadir selector de tema claro/oscuro.	Pablo	Interesante	4	1	—
DOC422	Registrar incidencias en acta diaria y su resolución.	Paula	Importante	6	1	TES412, TES413, TES416, TES433, TES440
RNF423	Revisar rutas, permisos y protección de ficheros.	Daniel	Imprescindible	5	1	—
BD424	Asegurar que cada acción se refleje en historial.json.	Franco	Importante	5	1	—
RNF425	Implementar limpieza automática del historial por tamaño.	Pablo	Importante	5	1	—
RNF426	Generar versión lista para distribución o demo.	Paula	Interesante	7	1	—
RNF427	Revisar librerías y versiones instaladas.	Daniel	Importante	6	1	—
BD428	Generar exportación de logs en formato JSON final.	Franco	Imprescindible	6	1	—



Cód.	Detalles	Persona Asignada	Prioridad	Inicio (día)	Duración (días)	Dependencias
RF429	Evaluar uso de CPU/RAM durante simulación prolongada.	Pablo	Importante	6	1	—
RNF430	Incluir panel de resumen de sensores y eventos totales.	Paula	Interesante	8	1	—
RNF431	Validar el proyecto frente a los requisitos iniciales.	Daniel	Imprescindible	7	1	—
RNF432	Revisar dependencias circulares entre scripts.	Franco	Importante	7	1	—
TES433	Eliminar elementos de debug o botones de prueba.	Pablo	Importante	7	1	BD424, BD428, RF406, RF429, RF436
DOC434	Completar acta y registrar definición de hecho alcanzada.	Paula	Interesante	9	1	TES412, TES413, TES416, TES433, TES440
RNF435	Coordinar presentación final con feedback del profesor.	Daniel	Imprescindible	8	1	—
RF436	Comprimir registros de simulación tras cierre.	Franco	Importante	8	1	—
DOC437	Redactar guía breve de uso para presentación.	Pablo	Interesante	8	1	TES412, TES413, TES416, TES433, TES440
DOC438	Confirmar que todos los documentos se suben correctamente.	Paula	Importante	10	1	TES412, TES413, TES416, TES433, TES440

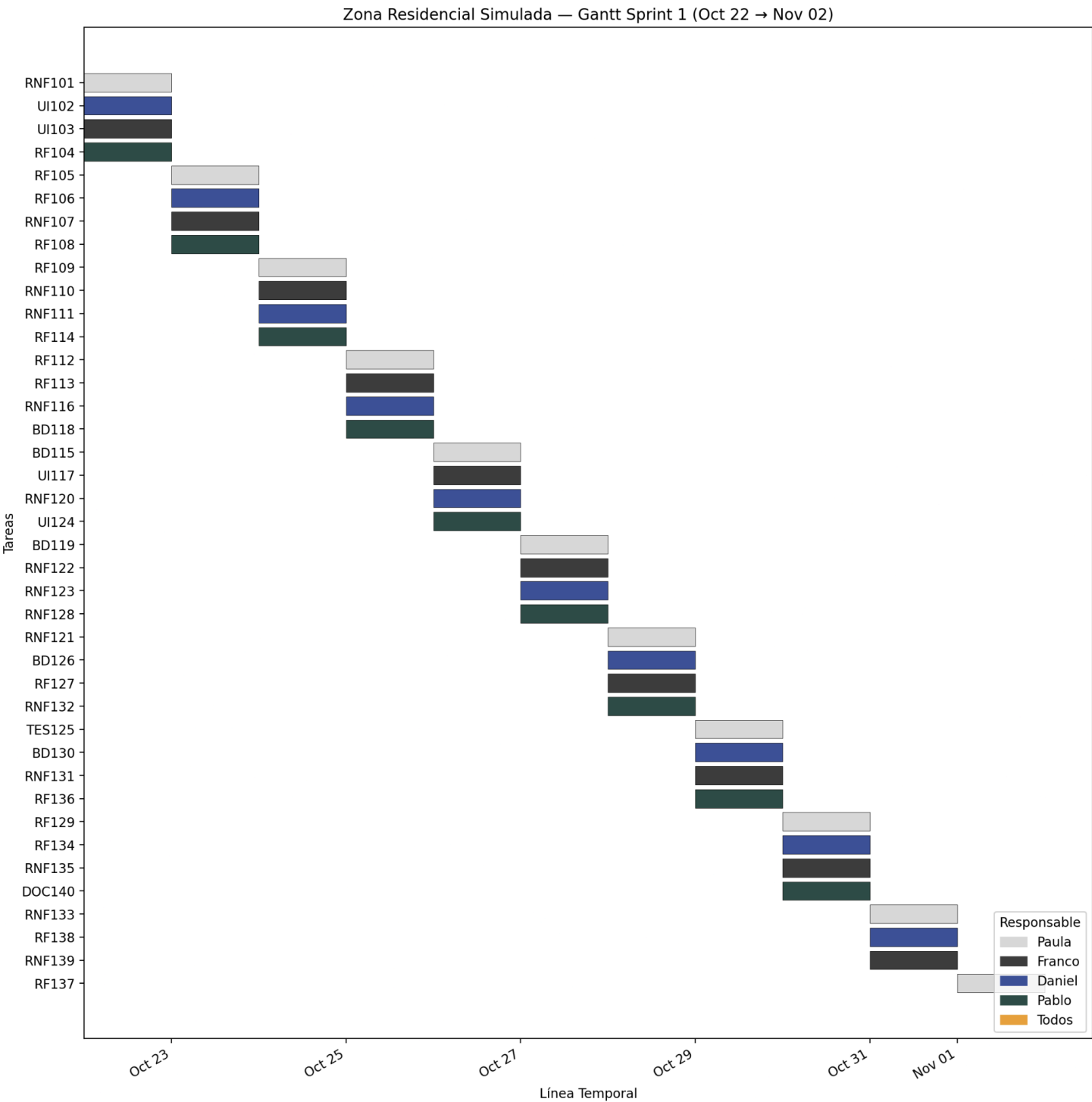


Cód.	Detalles	Persona Asignada	Prioridad	Inicio (día)	Duración (días)	Dependencias
DOC439	Redactar cierre formal y conclusiones finales.	Daniel	Imprescindible	9	1	TES412, TES413, TES416, TES433, TES440
TES440	Realizar code review y validación final antes de la presentación.	Todos	Importante	0	1	BD424, BD428, RF406, RF429, RF436

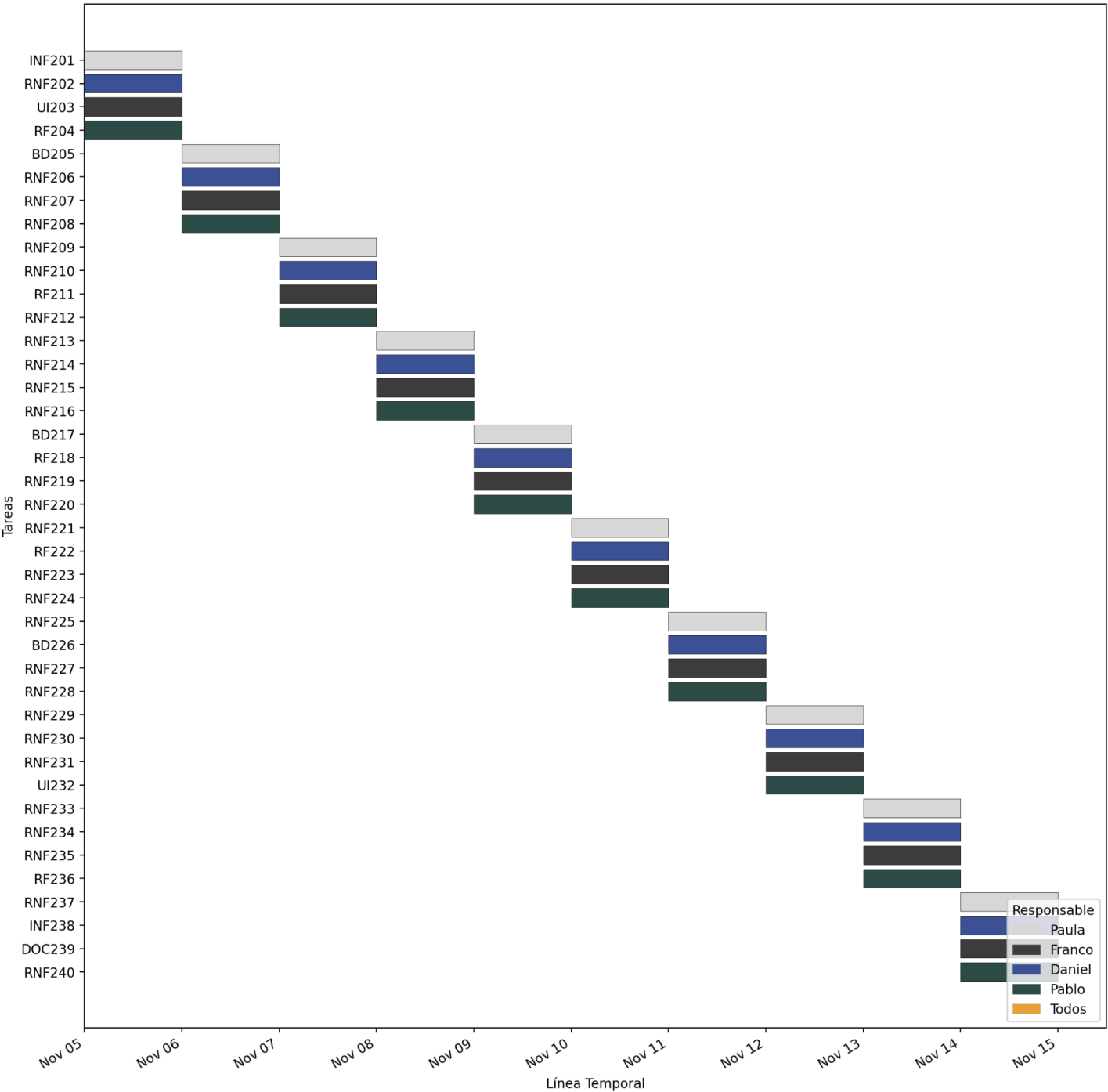
En conjunto, esta planificación refleja la distribución estructurada y equilibrada del trabajo a lo largo de los distintos Sprints, garantizando una evolución progresiva del sistema desde la configuración inicial hasta la integración final. La secuenciación temporal de las tareas, junto con la definición de dependencias y prioridades, permite optimizar los recursos disponibles y mantener la coherencia entre las fases de desarrollo, validación y documentación. Asimismo, la organización del trabajo mediante la metodología SCRUM asegura la entrega incremental de resultados funcionales, la revisión continua del progreso y la adaptación iterativa del proyecto a las necesidades detectadas en cada sprint. De esta forma, se garantiza un control efectivo del avance y una trazabilidad completa del proceso de ingeniería seguido por el equipo.

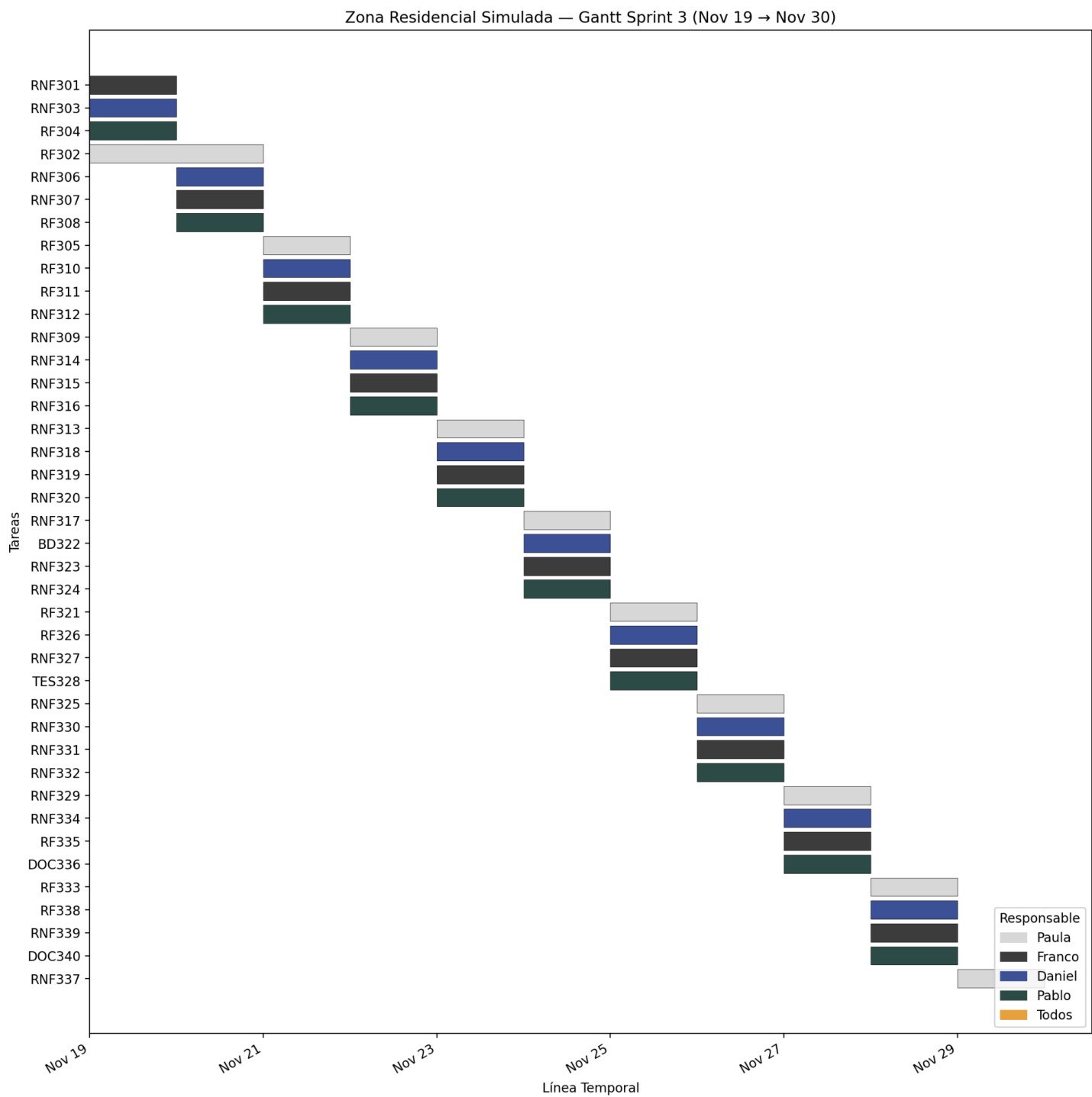


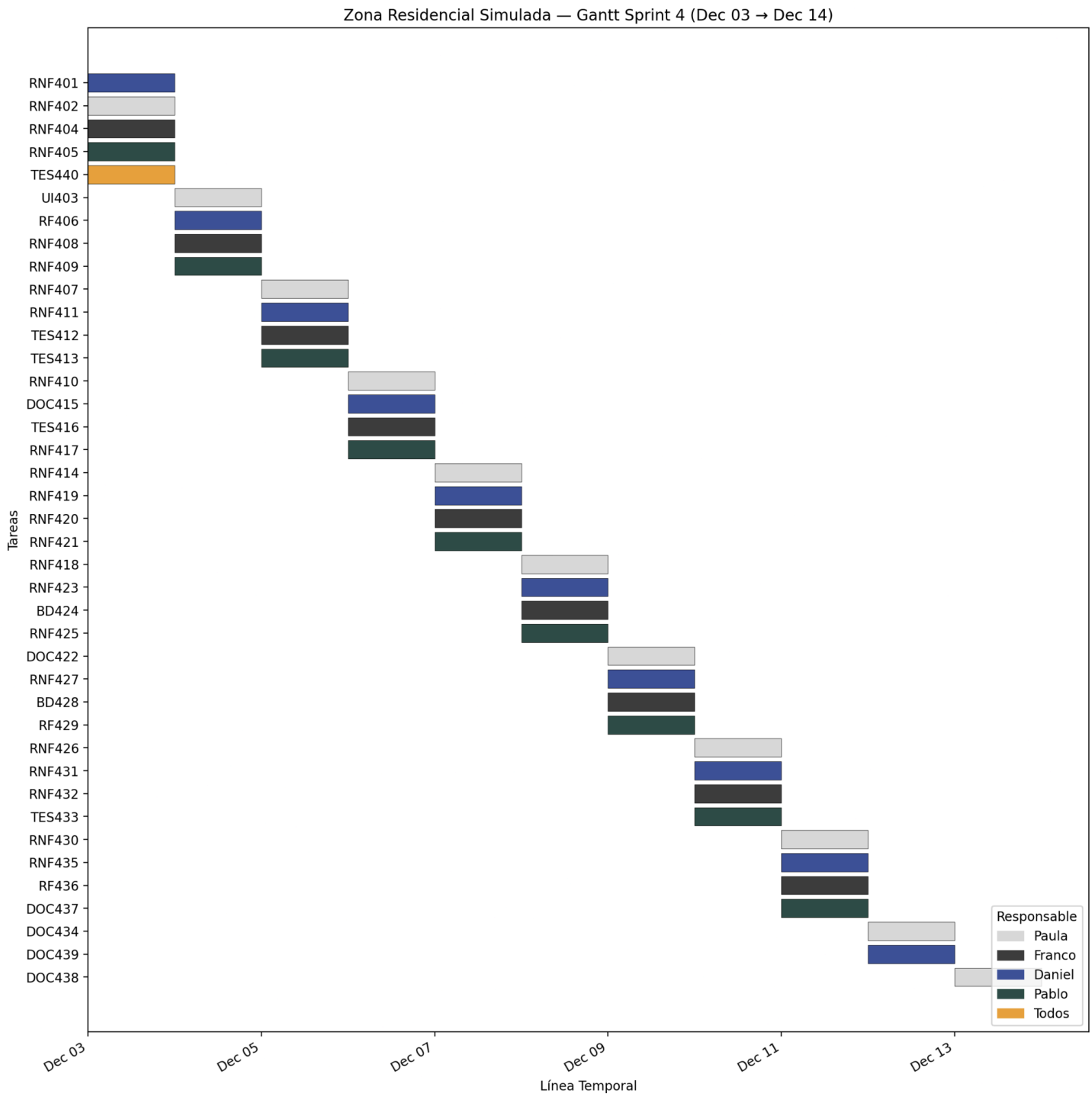
2.5. Diagrama de Gantt



Zona Residencial Simulada — Gantt Sprint 2 (Nov 05 → Nov 16)







2.6. Presupuesto

Debido a la naturaleza estimativa del cálculo de horas y a posibles variaciones derivadas del ritmo de trabajo y la coordinación entre los miembros del equipo, se ha considerado un margen de error del 10% sobre



las horas planificadas. Este margen permite reflejar con mayor realismo las posibles desviaciones en la ejecución del proyecto.

Los costes dependiendo del puesto que desempeña cada empleado se han calculado utilizando una media de sueldos para trabajadores con entre uno y tres años de experiencia en ese campo en España y el coste adicional que supone pertenecer a la sociedad, siendo todos los trabajadores autónomos miembros de la sociedad.

A continuación se muestra el (I) cálculo de coste para la empresa por puesto, (II) estimación de horas necesarias por sprint hasta la finalización del proyecto, (III) coste por sprint por empleado, (IV) coste del material y (V) coste total.

TABLA [I]

COSTE PARA LA EMPRESA EN FUNCIÓN DEL PUESTO

Puesto	Salario bruto (€)	Coste empresa (€)	Coste hora (€)
Product Owner	€38.000	€42.500	€25,30
Scrum Master	€36.000	€40.300	€23,99
Developer	€32.000	€35.800	€21,31

El coste por empleado para la empresa se calcula añadiendo al salario un 12% del mismo teniendo en cuenta que es una startup y cada miembro es un autónomo dentro de la sociedad que ha de pagar las cuotas fijas de la Seguridad social.

TABLA [II]

ESTIMACIÓN DE TIEMPO NECESARIO POR SPRINT HASTA LA FINALIZACIÓN DEL PROYECTO



Sprint	Product Owner(h)	Scrum Master (h)	Dev 1 (h)	Dev 2 (h)	Total horas (h)
Sprint 1	20	22	19	19	80
Sprint 2	20	19	22	19	80
Sprint 3	19	22	20	19	80
Sprint 4	19	20	19	22	80
Total horas por rol	78	83	80	79	320

TABLA [III]

COSTE POR SPRINT DIFERENCIANDO POR ROL DENTRO DEL PROYECTO

Sprint	Product Owner	Scrum Master	Dev 1	Dev 2	Total Sprint
Sprint 1	€505,95	€527,74	€404,88	€404,88	€1.843,45
Sprint 2	€505,95	€455,77	€468,81	€404,88	€1.835,42
Sprint 3	€480,65	€527,74	€426,19	€404,88	€1.839,46
Sprint 4	€480,65	€479,76	€404,88	€468,81	€1.834,11
Total por rol	€1.973,21	€1.991,01	€1.704,76	€1.683,45	€7.352,44

TABLA [IV]

COSTE DEL MATERIAL



Concepto	Precio (€)	Uds	Subtotal
Sensor temp/hum DHT22	1,00 €	1	1,00 €
Fotorresistor(Ldr)	1,00 €	1	1,00 €
Sensor ultrasonidos(HC-SR04)	1,00 €	1	1,00 €
Total			3,00 €

TABLA [V]

COSTE TOTAL DEL PROYECTO

Rol	Total Horas	Coste Hora (€)	Coste Total (€)
Product Owner	78	€25,30	€1.973,21
Scrum Master	83	€23,99	€1.991,01
Dev 1	80	€21,31	€1.704,76
Dev 2	79	€21,31	€1.683,45
+ Coste del material			€3,00
Coste Total Proyecto			€7.355,44



3. Diseño del proyecto

3.1. Prototipo

 APP Escritorio AureaTech



4. Resultados esperados

El desarrollo del sistema Zona Residencial Simulada se espera culmine con la entrega de una **aplicación funcional, modular y escalable**, capaz de integrar en un entorno unificado la monitorización y el control de sensores distribuidos en una zona residencial simulada.

Desde el punto de vista **técnico**, el proyecto deberá proporcionar:

- Una **arquitectura estable basada en el patrón MVC**, que garantice la separación lógica entre los componentes de presentación, control y gestión de datos.
- Una **interfaz desarrollada en Flet** coherente con la identidad visual de AureaTech, adaptativa a distintas resoluciones y con navegación intuitiva entre los paneles de usuario.
- Un **sistema de registro, autenticación y roles** completamente operativo, que permita la diferenciación de vistas entre administrador, técnico y vecino.
- Un **mecanismo de adquisición y simulación de datos** que reproduzca el comportamiento real de los sensores DHT22, LDR y HC-SR04, almacenando lecturas periódicas y su histórico en ficheros JSON.
- Un **panel de control administrativo** con funciones CRUD para la gestión de usuarios, sensores y actuadores, así como visualización en tiempo real del estado del sistema.
- Un **panel técnico** que muestre registros de eventos, logs de funcionamiento y estadísticas agregadas de temperatura, humedad y luminosidad.
- Un **panel de usuario vecino** con acceso simplificado a la información esencial y a alertas del sistema.

En términos de **rendimiento y usabilidad**, se espera:

- Una **respuesta fluida y sin bloqueos**, con actualización de datos en intervalos configurables y manejo eficiente de los recursos locales.
- **Persistencia fiable** de la información en los archivos JSON y mecanismos de control de errores y validación de datos.



- **Experiencia de usuario homogénea y accesible**, mediante elementos visuales claros, colores adaptativos y mensajes de retroalimentación consistentes.

Desde la perspectiva **organizativa y metodológica**, se prevé:

- La **culminación satisfactoria de los cuatro Sprints** definidos en la planificación, con entregas incrementales verificadas mediante pruebas funcionales.
- La **documentación completa del proceso**, incluyendo actas, informes técnicos y guía de uso final.
- La **presentación de un producto final integrable y demostrable**, preparado para su despliegue o simulación en un entorno educativo.

Adicionalmente, en caso de que el calendario de desarrollo y los recursos lo permitan, se espera alcanzar **resultados opcionales** que aporten un valor añadido al sistema, tales como:

- La **implementación de un sistema de alarma de incendios**, con detección temprana mediante sensores de humo y emisión de avisos automáticos.
- El **diseño de una puerta automática de garaje con apertura mediante reconocimiento de matrículas**, utilizando el sensor OV2640 como módulo de visión.
- La **integración de un anemómetro digital** para la medición de la velocidad del viento y su visualización dentro del panel técnico.

Estos desarrollos opcionales reforzarían el carácter **escalable y versátil** del proyecto, consolidando la propuesta de AureaTech como una solución completa de **automatización y control inteligente para entornos residenciales simulados**, capaz de evolucionar hacia un sistema IoT plenamente funcional.



5. Referencias

1. Reptil.mx, “Fotoresistor,” Industrias GSL, 2025.
https://industriasgsl.com/blogs/automatizacion/fotoresistor?srsId=AfmBOorZ-z0d8McZgOpyhp_wjwr0a5kAJcFtC9Aogiqc-UCdGmAl4cf5
(accessed Oct. 01, 2025).
2. “Sensor DHT22 Temperatura y Humedad con Cable | Arduino y ESP32,” CREATEC4 S.C.A., 2025.
<https://createc3d.com/tienda/es/modulos-y-sensores/1109-sensor-digit-al-de-humedad-y-temperatura-dht22.html> (accessed Oct. 01, 2025).
3. “Sensor Ultrasonido HC-SR04,” Naylamp Mechatronics – Perú.
<https://naylampmechatronics.com/sensores-proximidad/10-sensor-ultrasonido-hc-sr04.html> (accessed Oct. 01, 2025).

